

实验一：基于卷积神经网络的 MNIST 手写数字分类

朱立景 2025E8013282177

一、概述

本实验旨在利用卷积神经网络（Convolutional Neural Network, CNN）对 MNIST 手写数字数据集进行 0-9 共 10 类分类。MNIST 是深度学习领域最经典的入门基准数据集之一，任务目标是在测试集上达到较高的分类准确率。

任务定义： 给定一张 28×28 的灰度手写数字图像，预测其对应的数字类别（0-9）。

数据集： MNIST，包含 60,000 张训练图像和 10,000 张测试图像，共 10 个类别，每类样本分布均衡。

解决方案： 设计一个由两层卷积特征提取模块和全连接分类头组成的 CNN 网络，采用交叉熵损失函数与 Adam 优化器进行端到端训练，并配合 BatchNorm、Dropout、数据增强与学习率衰减等策略提升泛化能力。

实验目标： 测试集分类准确率达到 98% 及以上。

运行环境： Conda, Python 3.10, PyTorch 2.5.1 + CUDA, NVIDIA A800 GPU。

二、解决方案

2.1 网络结构设计

网络整体采用「特征提取 + 分类头」的两段式结构，参数量约 167 万。特征提取部分由两个卷积块组成，每个卷积块包含两个 ConvBlock（卷积 + 批归一化 + ReLU）、一次最大池化和一次 Dropout；分类头将特征图展平后，经 512 维全连接层映射到 10 类输出。

特征图尺寸变化：

```
1  输入：1×28×28
2    → ConvBlock×2：32×28×28
3    → MaxPool：    32×14×14
4    → ConvBlock×2：64×14×14
5    → MaxPool：    64×7×7
6    → Flatten：    3136
7    → FC(512) → FC(10)
```

核心代码 — ConvBlock 与 MNISTCNN：

```
1  class ConvBlock(nn.Module):
2      """Conv2d -> BatchNorm2d -> ReLU"""
3      def __init__(self, in_ch, out_ch, kernel=3, padding=1):
4          super().__init__()
5          self.block = nn.Sequential(
6              nn.Conv2d(in_ch, out_ch, kernel_size=kernel, padding=padding, bias=False),
7              nn.BatchNorm2d(out_ch),
8              nn.ReLU(inplace=True),
9          )
10
11 class MNISTCNN(nn.Module):
12     def __init__(self):
13         super().__init__()
14         self.features = nn.Sequential(
15             ConvBlock(1, 32), ConvBlock(32, 32),
```

```
16         nn.MaxPool2d(2), nn.Dropout2d(0.25),
17         ConvBlock(32, 64), ConvBlock(64, 64),
18         nn.MaxPool2d(2), nn.Dropout2d(0.25),
19     )
20     self.classifier = nn.Sequential(
21         nn.Flatten(),
22         nn.Linear(64 * 7 * 7, 512),
23         nn.BatchNorm1d(512), nn.ReLU(inplace=True),
24         nn.Dropout(0.5),
25         nn.Linear(512, 10),
26     )
```

设计要点：

组件	作用
3×3 卷积 + padding=1	保持空间分辨率，逐层提取局部纹理特征
BatchNorm	加速收敛，缓解内部协变量偏移
MaxPool2d(2)	降采样，扩大感受野，减少参数量
Dropout2d(0.25) / Dropout(0.5)	抑制过拟合，增强泛化
bias=False（卷积层）	与 BatchNorm 配合，减少冗余参数

2.2 损失函数设计

采用多分类任务中最常用的**交叉熵损失函数**（CrossEntropyLoss）。该损失函数内部集成了 Softmax 与负对数似然（NLL），直接对网络输出的 logits 与真实标签计算损失，数值稳定且梯度表达清晰。

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{z_{y_i}}}{\sum_{j=0}^9 e^{z_j}}$$

其中 z 为网络输出 logits， y_i 为第 i 个样本的真实类别。

核心代码：

```
1 criterion = nn.CrossEntropyLoss()
2 logits = model(images)
3 loss = criterion(logits, labels)
4 loss.backward()
```

2.3 优化器设计

采用 **Adam 优化器**，初始学习率设为 `1e-3`，兼顾收敛速度与训练稳定性。同时配合 **StepLR 学习率调度器**，每 5 个 epoch 将学习率衰减为原来的 0.5 倍，使训练后期能以更小的步长精细调整参数。

超参数	取值
优化器	Adam
初始学习率	1e-3
学习率调度	StepLR(step_size=5, gamma=0.5)
Batch Size	128
训练轮数	12

核心代码：

```
1 optimizer = optim.Adam(model.parameters(), lr=1e-3)
2 scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)
3 # 每个 epoch 结束后
4 scheduler.step()
```

2.4 数据预处理

- **训练集：** 随机旋转 $\pm 10^\circ$ 数据增强 $\rightarrow$ 转 Tensor $\rightarrow$ 按 MNIST 官方均值/标准差归一化
- **测试集：** 仅做 Tensor 转换与归一化，不做增强

```
1 train_tf = transforms.Compose([
2     transforms.RandomRotation(10),
3     transforms.ToTensor(),
4     transforms.Normalize((0.1307,), (0.3081,)),
5 ])
```

2.5 创新点与工程优化

本实验在经典 CNN 架构基础上做了以下设计与优化：

1. **模块化 ConvBlock：** 将「Conv $\rightarrow$ BN $\rightarrow$ ReLU」封装为可复用模块，结构清晰、便于扩展。
2. **分层 Dropout 策略：** 卷积层使用 Dropout2d(0.25)，全连接层使用 Dropout(0.5)，针对不同层级采用不同强度的正则化。
3. **轻量数据增强：** 仅对训练集施加 $\pm 10^\circ$ 随机旋转，在不显著改变数字形态的前提下提升模型对旋转扰动的鲁棒性。
4. **最优模型保存：** 训练过程中持续跟踪测试集准确率，自动保存表现最好的 checkpoint，避免过拟合后期性能回退的影响。

三、实验分析

3.1 数据集介绍

MNIST（Modified National Institute of Standards and Technology）手写数字数据集由 Yann LeCun 等人整理发布，是计算机视觉与深度学习领域使用最广泛的基准数据集之一。

属性	说明
图像尺寸	28 × 28 像素，单通道灰度
训练集	60,000 张
测试集	10,000 张
类别数	10（数字 0-9）
类别分布	各类约 6,000（训练） / 1,000（测试），均衡
像素值范围	0-255（归一化后约 [-0.42, 2.82]）
存储路径	<code>./data/MNIST/</code>

MNIST 任务难度适中：数字笔画简单、背景干净、类别均衡，适合验证 CNN 的基本分类能力，也是衡量模型是否「学会卷积特征提取」的标准试金石。

3.2 实验结果

在PyTorch 2.5.1 + CUDA, NVIDIA A800 GPU环境下完成 12 个 epoch 的训练，完整日志如下：

```
1  设备: cuda
2  参数量: 1,677,482
3  -----
4  Epoch 01/12 | train_loss=0.1549 | test_loss=0.0305 | test_acc=99.01% | 8.7s
5  Epoch 02/12 | train_loss=0.0636 | test_loss=0.0218 | test_acc=99.24% | 7.3s
6  Epoch 03/12 | train_loss=0.0511 | test_loss=0.0228 | test_acc=99.24% | 7.1s
7  Epoch 04/12 | train_loss=0.0444 | test_loss=0.0166 | test_acc=99.38% | 7.6s
8  Epoch 05/12 | train_loss=0.0388 | test_loss=0.0176 | test_acc=99.38% | 7.2s
9  Epoch 06/12 | train_loss=0.0307 | test_loss=0.0138 | test_acc=99.55% | 7.2s
10 Epoch 07/12 | train_loss=0.0253 | test_loss=0.0122 | test_acc=99.60% | 7.3s
11 Epoch 08/12 | train_loss=0.0253 | test_loss=0.0139 | test_acc=99.49% | 7.3s
12 Epoch 09/12 | train_loss=0.0248 | test_loss=0.0126 | test_acc=99.55% | 7.2s
13 Epoch 10/12 | train_loss=0.0228 | test_loss=0.0132 | test_acc=99.54% | 7.3s
14 Epoch 11/12 | train_loss=0.0193 | test_loss=0.0120 | test_acc=99.52% | 7.2s
15 Epoch 12/12 | train_loss=0.0175 | test_loss=0.0112 | test_acc=99.58% | 7.5s
16 -----
17 最佳测试准确率: 99.60%
18 模型已保存: /root/code/DL/exp1/checkpoints/mnist_cnn.pt
19 已达到目标 (>= 98%)
```

3.3 结果分析

(1) 收敛速度

模型在第 1 个 epoch 结束时测试准确率即达到 **99.01%**，已超过 98% 的实验目标，说明网络结构对 MNIST 任务是充分有效的。前 4 个 epoch 训练损失从 0.1549 快速下降至 0.0444，体现出 Adam 优化器与 BatchNorm 的良好配合。

(2) 最佳性能

最佳测试准确率为 **99.60%**（第 7 个 epoch），对应测试损失 0.0122。在 10,000 张测试图像中仅约 40 张分类错误，表现优异。

(3) 过拟合与泛化

训练过程中训练损失持续下降（0.1549 → 0.0175），而测试准确率在 99.49%-99.60% 之间小幅波动，未出现测试性能明显恶化的过拟合现象。Dropout 与数据增强起到了有效的正则化作用。第 5 个 epoch 后学习率减半，测试损失进一步降低，验证了学习率调度策略的有效性。

(4) 训练效率

单个 epoch 耗时约 7-9 秒，12 个 epoch 总训练时间约 90 秒，在 GPU 上效率较高。

(5) 与目标对比

指标	目标	实际结果
测试集准确率	$\geq 98\%$	99.60%
是否达标	—	是

四、总结

本实验成功设计并实现了一个面向 MNIST 手写数字分类的卷积神经网络。通过两层卷积特征提取（ConvBlock + MaxPool + Dropout）与全连接分类头的组合，配合交叉熵损失、Adam 优化器、学习率衰减及轻量数据增强，在 12 个 epoch 内将测试集准确率提升至 **99.60%**，显著超过 98% 的实验目标。

实验表明，对于 MNIST 这类结构简单的灰度图像分类任务，一个参数量约 167 万、结构规范的 CNN 已足以取得接近 99.6% 的高精度。BatchNorm 与 Dropout 的引入有效平衡了收敛速度与泛化能力，模块化 ConvBlock 设计也使网络结构清晰、易于理解与扩展。